Section 1 and Section 2: OOP Principles Through an Interview Lens

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 9: Section Intro: OOP Principles Through an Interview Lens

SLIDE 10: Encapsulation: Information Hiding as a Design Decision

SLIDE 11: Why Getters and Setters Are Not Encapsulation

SLIDE 12: Worked Example: User Account with Password Handling

SLIDE 13: Abstraction: Designing Clean Interfaces

SLIDE 14: Abstraction Leaks and How They Show Up in Interviews

SLIDE 15: Worked Example: Payment Processing Interface

SLIDE 15B: Interview Practice: Encapsulation and Abstraction

SLIDE 16: Inheritance: When It Works and When It Creates Coupling

SLIDE 17: The Fragile Base Class Problem

SLIDE 17B: Fragile Base Class: Concrete Example and Fix

SLIDE 18: Polymorphism: Runtime vs Compile-Time

SLIDE 19: Designing for Polymorphic Behavior

SLIDE 19B: Worked Example: Notification Channel Dispatch

SLIDE 19C: Interview Practice: Inheritance and Polymorphism

SLIDE 20: Composition vs Inheritance: The Core OOD Trade-Off

SLIDE 21: Where Inheritance Breaks and Composition Saves You

SLIDE 21B: Composition vs Inheritance: Decision Framework

SLIDE 22: Abstract Classes vs Interfaces

SLIDE 22B: Abstract Classes vs Interfaces: Decision Framework

SLIDE 22C: Note It Down: Key OOP Design Decisions

SLIDE 23: OOP Interview Practice and Common Mistakes

SLIDE 24: Section Summary and Cheat Sheet

SLIDE 9: Section Intro: OOP Principles Through an Interview Lens

Section 2: OOP Principles Through an Interview Lens

OOP is not a checklist of keywords to recite in an interview. Interviewers evaluate whether you can use OOP principles to make deliberate design decisions that produce clean, extensible, and maintainable class structures. This section covers each principle with the depth and practical framing that LLD interviews demand at mid-level and senior levels.

What We Will Cover

1. Encapsulation as information hiding, not access modifiers
2. Abstraction and clean interface design
3. Inheritance and its coupling risks
4. Polymorphism for extensible behavior
5. Composition vs Inheritance, the most important OOD trade-off
6. Abstract Classes vs Interfaces with a decision framework

Where OOP Appears in LLD Interviews

Interview Stage	How OOP Is Evaluated
Requirements Clarification	Can you identify the right objects and assign clear responsibilities?
Class Design	Do your classes have well-defined boundaries and clean interfaces?

Interview Stage	How OOP Is Evaluated
Extensibility Probe	"How would you add a new type of X?" Tests polymorphism and open-closed thinking
Trade-off Discussion	"Why composition here instead of inheritance?" Tests design maturity
Code Walkthrough	Are classes encapsulated properly? Do methods express behavior, not data access?

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 10: Encapsulation: Information Hiding as a Design Decision

Encapsulation means that a class controls access to its own internal state and exposes only what other classes need to interact with it. The key word is "controls." Encapsulation is a design decision about what to hide and what to reveal, not simply a matter of marking fields as private.

When you encapsulate properly, you protect invariants. An invariant is a condition that must always be true for the object to remain in a valid state. For example, in a banking system, an account balance should never go negative. Encapsulation ensures that no external code can violate this rule by directly modifying the balance field.

Encapsulation Protects Invariants

```
public class BankAccount {
    private double balance;
    private final String ownerName;
    private final List<Transaction> transactions = new ArrayL
ist<>();

    public BankAccount(String ownerName, double initialBalanc
e) {
        this.ownerName = ownerName;
        this.balance = initialBalance;
```

```
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal am
ount must be positive");
        }
        if (amount > balance) {
            throw new InsufficientFundsException(balance, amo
unt);
        }
        balance -= amount;
        transactions.add(new Transaction("withdrawal", amoun
t));
    }

    public double getBalance() {
        return balance;
    }
}
```

There is no `setBalance()` method. The only way to change the balance is through `withdraw()` or a corresponding `deposit()` method, both of which enforce business rules. External code cannot set the balance to an arbitrary value, which means the invariant (balance >= 0) is always maintained by the class itself.

Interview Relevance: Interviewers check whether your classes expose raw state or enforce rules through behavior methods. A setter on a field that has constraints is a design red flag.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 11: Why Getters and Setters Are Not Encapsulation

A common misconception is that making fields private and adding getters and setters for every field counts as encapsulation. It does not. If every internal field has a public getter and setter, the class is exposing its entire internal structure. You have added boilerplate without gaining any protection.

Common Mistake: Exposing Everything

```
public class UserProfile {
    private String name;
    private String email;
    private String password;

    public String getPassword() {
        return password;
    }

    public void setPassword(String newPassword) {
        this.password = newPassword;
    }
}
```

This class exposes the raw password through a getter and accepts any string through a setter. There is no validation, no hashing, no enforcement of password policy. The `private` keyword is cosmetic.

Stronger Approach: Behavior Over Data Access

```
public class UserProfile {
    private final String name;
    private final String email;
    private String passwordHash;

    public UserProfile(String name, String email, String rawP
assword) {
        this.name = name;
        this.email = email;
        this.passwordHash = PasswordHasher.hash(rawPassword);
```

```
    }

    public boolean auth3enticate(String rawPassword) {
        return PasswordHasher.verify(rawPassword, this.passwo
rdHash);
    }

    public void changePassword(String currentPassword, String
newPassword) {
        if (!authenticate(currentPassword)) {
            throw new AuthenticationException("Current passwo
rd is incorrect");
        }
        if (newPassword.length() < 8) {
            throw new ValidationException("Password must be a
t least 8 characters");
        }
        this.passwordHash = PasswordHasher.hash(newPassword);
    }
}
```

The password is never exposed. The class provides `authenticate()` and `changePassword()` as behavior. External code tells the object what to do, not how to store its data. This is encapsulation.

Interview Relevance: When an interviewer says "encapsulate this properly," they are checking whether you expose behavior or data. A class full of getters and setters for every field is a data bag, not an encapsulated object.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 12: Worked Example: User Account with Password Handling

This slide shows a properly encapsulated `UserAccount` class that manages authentication state. Notice how every method enforces a business rule. No field is directly accessible, and no state transition can bypass validation.

```
public class UserAccount {
    private final String userId;
    private String email;
    private String passwordHash;
    private int failedLoginAttempts;
    private boolean locked;

    public UserAccount(String userId, String email, String ra
wPassword) {
        this.userId = userId;
        this.email = email;
        this.passwordHash = PasswordHasher.hash(rawPassword);
        this.failedLoginAttempts = 0;
        this.locked = false;
    }

    public LoginResult authenticate(String rawPassword) {
        if (locked) {
            return LoginResult.accountLocked();
        }
        if (!PasswordHasher.verify(rawPassword, passwordHas
h)) {
            failedLoginAttempts++;
            if (failedLoginAttempts >= 5) {
                locked = true;
            }
            return LoginResult.invalidCredentials(5 - failedL
oginAttempts);
        }
        failedLoginAttempts = 0;
        return LoginResult.success(userId);
    }
```

```
    public void changeEmail(String newEmail, String currentPa
ssword) {
        if (!PasswordHasher.verify(currentPassword, passwordH
ash)) {
            throw new AuthenticationException("Password requi
red to change email");
        }
        this.email = newEmail;
    }
}
```

What Makes This Encapsulated

The `failedLoginAttempts` and `locked` fields have no getters or setters. External code cannot unlock an account by calling `setLocked(false)`. The only way to interact with login state is through `authenticate()`, which enforces the lockout policy internally. The `changeEmail()` method requires the current password, which means email changes cannot happen without identity verification. Every method is a behavior, not a data accessor.

Interview Tip: When building your class diagram in an LLD interview, ask yourself for each field: "Should external code be able to read this? Should external code be able to change this? Under what conditions?" If the answer to any of those requires a rule, that field needs a behavior method, not a setter.

Interview Relevance: Interviewers evaluate the ratio of behavior methods to getters/setters in your class design. Classes with mostly behavior methods signal that you understand encapsulation as a design principle.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 13: Abstraction: Designing Clean Interfaces

Abstraction means hiding implementation complexity behind a simplified contract. While encapsulation hides data, abstraction hides how things work. A well-

designed abstraction lets calling code say what it wants without knowing how the work gets done.

In Java, abstraction is expressed through interfaces and abstract classes. The interface defines the "what" and the implementing class handles the "how."

Abstraction in Practice

```
public interface MessageSender {
    SendResult send(String recipient, String content);
    DeliveryStatus checkDelivery(String messageId);
}
```

Any class that processes messages depends on `MessageSender`, not on a specific implementation. The calling code does not know whether the message is sent via email, SMS, or push notification. It does not need to know.

```
public class OrderConfirmationService {
    private final MessageSender messageSender;

    public OrderConfirmationService(MessageSender messageSend
er) {
        this.messageSender = messageSender;
    }

    public void confirmOrder(Order order) {
        String content = buildConfirmationMessage(order);
        messageSender.send(order.getCustomerContact(), conten
t);
    }
}
```

`OrderConfirmationService` works with email today and SMS tomorrow without any code change. The abstraction boundary is the interface. Everything behind it can change freely.

The Difference Between Encapsulation and Abstraction

Principle	What It Hides	How It Is Expressed	What It Protects
Encapsulation	Internal data and state	Private fields, behavior methods	Invariants and business rules
Abstraction	Implementation complexity	Interfaces and abstract classes	Calling code from change

Both work together. A well-encapsulated class behind a clean interface gives you maximum flexibility: you can change both the internal data representation and the implementation approach without affecting any code outside the class.

Interview Relevance: Interviewers test abstraction by asking "what if we need to swap this component?" If swapping requires modifying existing classes, your abstraction is leaking.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 14: Abstraction Leaks and How They Show Up in Interviews

An abstraction leak occurs when implementation details escape through the interface, forcing calling code to know things it should not need to know. Abstraction leaks are subtle and interviewers specifically probe for them.

Common Abstraction Leak: Exposing Internal Data Structures

```
public class OrderRepository {
    private final Map<String, Order> orderStore = new HashMap
<>();

    public Map<String, Order> getOrderStore() {
        return orderStore;
    }
}
```

This exposes the internal `HashMap`. Calling code now depends on the fact that orders are stored in a `Map`. If you later switch to a database, every caller must

change. Worse, external code can call `orderStore.clear()` and wipe all data without going through any validation.

Fixed Version: Hide the Storage Mechanism

```
public class OrderRepository {
    private final Map<String, Order> orderStore = new HashMap
<>();

    public Optional<Order> findById(String orderId) {
        return Optional.ofNullable(orderStore.get(orderId));
    }

    public List<Order> findByCustomer(String customerId) {
        return orderStore.values().stream()
            .filter(order -> order.getCustomerId().equals(cus
tomerId))
            .collect(Collectors.toList());
    }

    public void save(Order order) {
        orderStore.put(order.getId(), order);
    }
}
```

The internal `HashMap` is completely hidden. Callers interact through `findById()`, `findByCustomer()`, and `save()`. Switching from an in-memory map to a PostgreSQL database changes only this class. Every caller remains untouched.

Interview Tip: A reliable test for abstraction leaks: if you changed the internal implementation (switched databases, changed data structures, swapped libraries), would any code outside this class need to change? If yes, you have a leak. Mention this test during your LLD interview and apply it to your own design.

Interview Relevance: Interviewers look for abstraction leaks in your class diagram. Returning raw collections, exposing internal types, or requiring callers to understand implementation details are all signals of a weak design.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 15: Worked Example: Payment Processing Interface

Consider a payment system that needs to support multiple providers: Stripe today, PayPal tomorrow, and a regional provider next quarter. Without abstraction, every class that processes payments would need to know the specific API details of each provider. Abstraction solves this by defining a clean contract that hides implementation complexity.

The Abstraction

```
public interface PaymentProcessor {
    PaymentResult charge(Money amount, PaymentMethod method);
    PaymentResult refund(String transactionId, Money amount);
    PaymentStatus checkStatus(String transactionId);
}
```

One Concrete Implementation

```
public class StripePaymentProcessor implements PaymentProcess
or {
    private final StripeClient stripeClient;

    public StripePaymentProcessor(StripeClient stripeClient)
{
        this.stripeClient = stripeClient;
    }

    @Override
    public PaymentResult charge(Money amount, PaymentMethod m
ethod) {
        StripeChargeRequest request = new StripeChargeRequest
(
            amount.toCents(), amount.getCurrencyCode(), metho
```

```
d.getTokenizedId()
        );
        StripeChargeResponse response = stripeClient.createCh
arge(request);
        return new PaymentResult(response.getId(), mapStatus
(response.getStatus()));
    }
}
```

The `OrderService` depends on `PaymentProcessor`, never on `StripePaymentProcessor` directly. Switching from Stripe to PayPal means writing a new `PayPalPaymentProcessor` class. Zero changes to `OrderService` or any other calling code. This is what abstraction gives you: the ability to change the "how" without affecting the "what."

Interview Tip: When designing class diagrams in an LLD interview, define your abstractions (interfaces) before your implementations. This signals to the interviewer that you think in terms of contracts and dependencies, not just concrete classes.

Interview Relevance: Interviewers frequently ask "what if we need to swap this component?" If your design requires modifying existing classes to add a new provider, your abstraction is leaking.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 15B: Interview Practice: Encapsulation and Abstraction

Scenario

You are designing a `NotificationPreferences` class for a system like LinkedIn. Users can configure which notification types they want (email, SMS, push) and set quiet hours during which no notifications are sent.

An interviewer asks: "Show me your class design for `NotificationPreferences`."

Pause the video and design this class for 60 seconds before continuing.

Weak Answer

```
public class NotificationPreferences {
    private Set<String> enabledChannels;
    private int quietStartHour;
    private int quietEndHour;

    public Set<String> getEnabledChannels() { return enabledC
hannels; }
    public void setEnabledChannels(Set<String> channels) { th
is.enabledChannels = channels; }
    public int getQuietStartHour() { return quietStartHour; }
    public void setQuietStartHour(int hour) { this.quietStart
Hour = hour; }
    public int getQuietEndHour() { return quietEndHour; }
    public void setQuietEndHour(int hour) { this.quietEndHour
= hour; }
}
```

This is a data bag. External code can set quiet hours to invalid values (start = 25, end = -3). The raw Set is exposed, so callers can add arbitrary strings like "fax" or "pigeon." There are no behavior methods. Every business rule must live in the calling code.

Strong Answer

```
public class NotificationPreferences {
    private final Set<NotificationChannel> enabledChannels =
new HashSet<>();
    private int quietStartHour;
    private int quietEndHour;

    public void enableChannel(NotificationChannel channel) {
        enabledChannels.add(channel);
    }

    public void disableChannel(NotificationChannel channel) {
        enabledChannels.remove(channel);
```

```
    }

    public boolean shouldNotify(NotificationChannel channel,
LocalTime currentTime) {
        if (!enabledChannels.contains(channel)) {
            return false;
        }
        return !isQuietHours(currentTime);
    }

    public void setQuietHours(int startHour, int endHour) {
        if (startHour < 0 || startHour > 23 || endHour < 0 ||
endHour > 23) {
            throw new IllegalArgumentException("Hours must be
between 0 and 23");
        }
        this.quietStartHour = startHour;
        this.quietEndHour = endHour;
    }
}
```

The strong answer uses an enum (`NotificationChannel`) instead of raw strings, validates quiet hours, and provides `shouldNotify()` as a behavior method. The calling code asks "should I notify?" instead of pulling out fields and making that decision itself.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 16: Inheritance: When It Works and When It Creates Coupling

Inheritance establishes an "is-a" relationship: a `SavingsAccount` is a `BankAccount`, a `Manager` is an `Employee`. The subclass inherits all fields and methods from the parent, and can override behavior to specialize it.

When Inheritance Works Well

Inheritance is appropriate when three conditions are all true. First, the relationship is genuinely "is-a", not "has-a" or "uses-a." Second, the subclass needs most or all of the parent class behavior, not just one or two methods. Third, the parent class is designed for inheritance with clear extension points.

```
public abstract class Shape {
    private final String color;

    public Shape(String color) {
        this.color = color;
    }

    public abstract double area();

    public String describe() {
        return color + " shape with area " + area();
    }
}

public class Circle extends Shape {
    private final double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

This works because `Circle` genuinely is a `Shape`, uses the inherited `describe()` method, and `Shape` was designed with the abstract `area()` method as a clear extension point.

When Inheritance Creates Problems

Inheritance creates tight coupling between parent and child. The subclass depends on the parent's implementation, not just its interface. If the parent changes how a method works internally, every subclass can break. The subclass also inherits everything, including methods it may not need or want. You cannot selectively inherit. It is all or nothing.

Interview Tip: If you use inheritance in your LLD answer, be prepared for the interviewer to ask "what happens when requirements change?" Interviewers often probe inheritance decisions because they want to see if you understand the coupling risks.

Interview Relevance: Using inheritance without justifying why it is the right choice (over composition) is one of the most common mistakes in LLD interviews. Always state your reasoning.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 17: The Fragile Base Class Problem

The fragile base class problem occurs when changes to a parent class break subclasses in ways that are impossible to predict from the parent class alone. This happens because the subclass depends on the parent's internal implementation, not just its public interface.

The root cause is self-use: when one method in the base class calls another method in the same class, and the subclass overrides one or both of those methods. The subclass author must understand exactly how the base class implements every method internally, which violates the principle of abstraction.

This problem is not theoretical. It appears in production codebases, in standard library design discussions, and in LLD interviews when an interviewer asks "what are the risks of this inheritance hierarchy?"

The next slide shows a concrete example of this problem and how to fix it with composition.

Interview Relevance: Interviewers use the fragile base class problem to test whether you understand inheritance risks beyond the textbook definition. Being able to explain this with a concrete example separates mid-level from senior candidates.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 17B: Fragile Base Class: Concrete Example and Fix

The Problem

```
public class EventLog {
    private final List<Event> events = new ArrayList<>();

    public void addEvent(Event event) {
        events.add(event);
    }

    public void addEvents(List<Event> newEvents) {
        for (Event event : newEvents) {
            addEvent(event);
        }
    }
}
```

```
public class CountingEventLog extends EventLog {
    private int eventCount = 0;

    @Override
    public void addEvent(Event event) {
        eventCount++;
```

```
        super.addEvent(event);
    }

    @Override
    public void addEvents(List<Event> newEvents) {
        eventCount += newEvents.size();
        super.addEvents(newEvents);
    }

    public int getEventCount() {
        return eventCount;
    }
}
```

Calling `addEvents()` with 3 events produces `eventCount = 6`. The base class `addEvents()` internally calls `addEvent()`, which the subclass has overridden. So each event gets counted twice: once in `addEvents()` and once in `addEvent()`. The subclass is coupled to an implementation detail of the base class that is not part of its public contract. If the base class author later changes `addEvents()` to use `events.addAll()` instead of looping through `addEvent()`, the counting subclass silently breaks in the opposite direction.

The Fix: Composition

```
public class CountingEventLog {
    private final EventLog delegate = new EventLog();
    private int eventCount = 0;

    public void addEvent(Event event) {
        eventCount++;
        delegate.addEvent(event);
    }

    public void addEvents(List<Event> newEvents) {
        eventCount += newEvents.size();
        delegate.addEvents(newEvents);
```

```
    }

    public int getEventCount() {
        return eventCount;
    }
}
```

Now `CountingEventLog` does not extend `EventLog`. It wraps it. The counting logic is completely independent of how `EventLog` implements `addEvents()` internally. If the base class changes its implementation tomorrow, the counting wrapper is unaffected.

Interview Relevance: If you use inheritance in your LLD answer, interviewers may probe with "what happens if the base class implementation changes?" Having the fragile base class problem in mind lets you justify when composition is the safer choice.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 18: Polymorphism: Runtime vs Compile-Time

Polymorphism means that the same method call can produce different behavior depending on the object it is called on. In Java, there are two forms.

Runtime Polymorphism (Method Overriding)

The actual method that runs is determined at runtime based on the real type of the object, not the declared type of the variable. This is the form that matters most for LLD interviews.

```
PaymentProcessor processor = new StripePaymentProcessor();
processor.charge(amount, method);
```

The variable type is `PaymentProcessor` (the interface), but the method that executes is `StripePaymentProcessor.charge()`. If you reassign the variable to a

`PayPalPaymentProcessor`, the same `processor.charge()` call executes completely different code. The calling code does not change.

Compile-Time Polymorphism (Method Overloading)

The method is selected at compile time based on the parameter types. This is useful but less important for design.

```
public class PriceCalculator {
    public Money calculate(Order order) {
        return order.getSubtotal().add(calculateTax(order));
    }

    public Money calculate(Order order, DiscountCode discoun
t) {
        Money subtotal = order.getSubtotal();
        Money discounted = discount.applyTo(subtotal);
        return discounted.add(calculateTax(order));
    }
}
```

Same method name, different parameter lists. The compiler picks the right version based on the arguments passed.

Why Runtime Polymorphism Matters More for Design

Aspect	Runtime (Overriding)	Compile-Time (Overloading)
Decided at	Runtime (JVM dispatch)	Compile time (compiler picks)
Enables	Extensibility, plug-in architectures	Convenience, API usability
Used for	Strategy, Factory, Observer patterns	Builder-style fluent APIs
Interview weight	Very high	Low

Interview Relevance: When interviewers say "design this to be extensible," they are asking you to use runtime polymorphism. They want to see that adding a new type requires writing a new class, not modifying existing code.

SLIDE 19: Designing for Polymorphic Behavior

Polymorphism does not happen by accident. You design for it by identifying the dimension of variation in your system and expressing it through an interface.

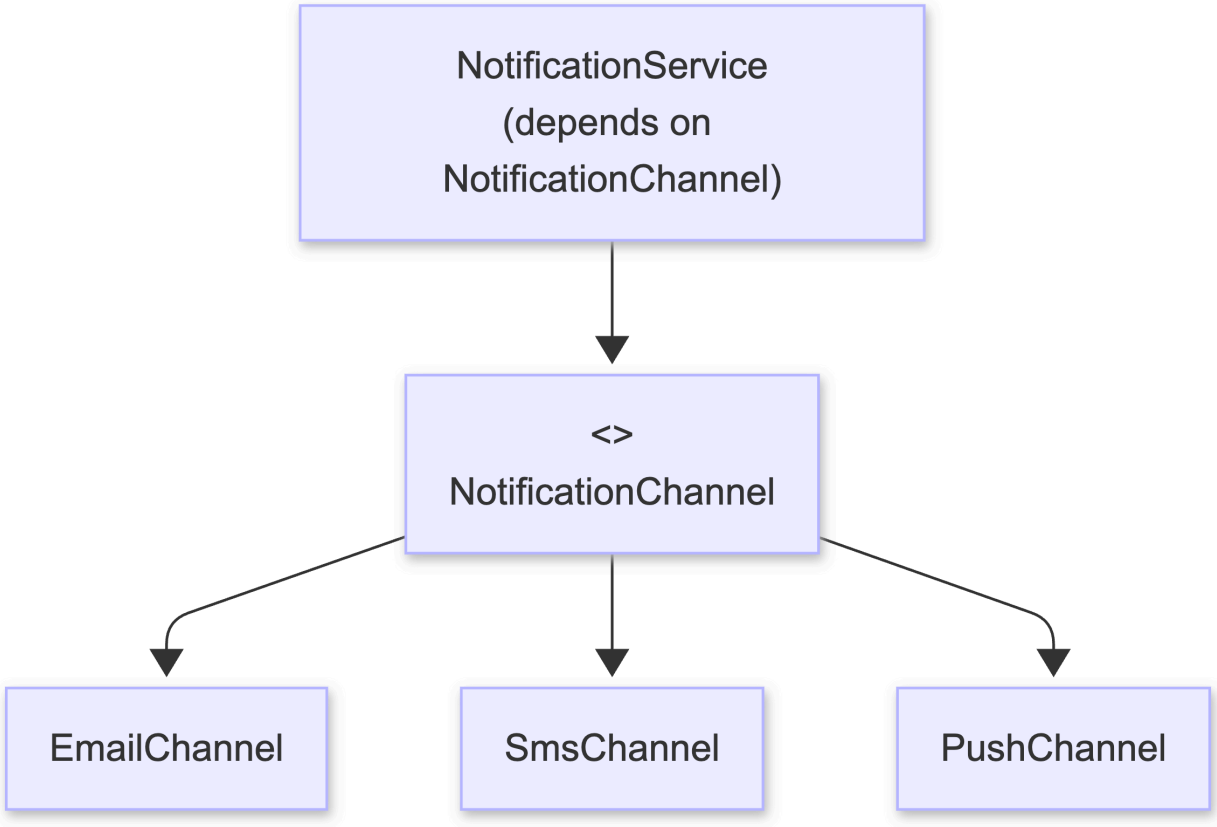
The Pattern

Step 1: Identify what varies. In a notification system, the delivery channel varies (email, SMS, push). The act of sending a notification does not vary.

Step 2: Define an interface for the varying behavior.

Step 3: Write one implementation per variant.

Step 4: Write calling code that depends on the interface, not any implementation.



The `NotificationService` calls `channel.send()` without knowing or caring which implementation is behind the interface. This is the foundation of extensibility. Adding WhatsApp as a channel means writing one new class. Zero changes to `NotificationService`.

Why This Matters

Without polymorphism, adding a new channel means modifying the `NotificationService` with another `if` or `switch` branch. With polymorphism, the service is closed for modification and open for extension. This is the Open-Closed Principle in action, which we will cover in depth in Section 3.

Interview Relevance: The ability to identify what varies and extract it into an interface is one of the strongest signals interviewers look for. It shows you can design systems that grow without becoming fragile.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 19B: Worked Example: Notification Channel Dispatch

Here is the full implementation of a polymorphic notification system for a platform like Slack.

The Interface

```
public interface NotificationChannel {
    void send(String userId, String message);
    boolean supports(NotificationType type);
}
```

Two Implementations

```
public class EmailChannel implements NotificationChannel {
    private final EmailClient emailClient;

    public EmailChannel(EmailClient emailClient) {
        this.emailClient = emailClient;
```

```
    }

    @Override
    public void send(String userId, String message) {
        String emailAddress = UserDirectory.lookupEmail(userI
d);
        emailClient.sendEmail(emailAddress, "Notification", m
essage);
    }

    @Override
    public boolean supports(NotificationType type) {
        return type == NotificationType.MARKETING
            || type == NotificationType.TRANSACTIONAL;
    }
}
```

```
public class PushChannel implements NotificationChannel {
    private final PushGateway pushGateway;

    public PushChannel(PushGateway pushGateway) {
        this.pushGateway = pushGateway;
    }

    @Override
    public void send(String userId, String message) {
        String deviceToken = DeviceRegistry.lookupToken(userI
d);
        pushGateway.sendPush(deviceToken, message);
    }

    @Override
    public boolean supports(NotificationType type) {
        return type == NotificationType.TRANSACTIONAL
            || type == NotificationType.ALERT;
```

```
    }
}
```

The Dispatcher

```
public class NotificationDispatcher {
    private final List<NotificationChannel> channels;

    public NotificationDispatcher(List<NotificationChannel> c
hannels) {
        this.channels = channels;
    }

    public void dispatch(String userId, String message, Notif
icationType type) {
        for (NotificationChannel channel : channels) {
            if (channel.supports(type)) {
                channel.send(userId, message);
            }
        }
    }
}
```

The dispatcher iterates through all registered channels and sends through every channel that supports the notification type. Adding WhatsApp requires writing a `WhatsAppChannel` class and registering it. The `NotificationDispatcher` class is never modified.

Interview Tip: When you present a polymorphic design in an interview, explicitly state: "Adding a new type of X requires writing one new class. No existing classes are modified." This directly demonstrates extensibility and signals design maturity to the interviewer.

Interview Relevance: This exact pattern (interface + multiple implementations + dispatcher) appears in nearly every LLD interview. Master it for notification systems, payment processing, file export, and discount calculation.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 19C: Interview Practice: Inheritance and Polymorphism

Scenario

You are designing a file export feature for a document management system like Google Docs. Users can export documents to PDF, DOCX, and HTML formats. The interviewer asks: "Show me your class design for the export feature."

Pause the video and design this for 60 seconds before continuing.

Weak Answer

```
public class DocumentExporter {
    public void export(Document doc, String format) {
        if (format.equals("pdf")) {
            PdfLibrary.render(doc.getContent(), doc.getStyles
());
        } else if (format.equals("docx")) {
            WordLibrary.convert(doc.getContent(), doc.getMeta
data());
        } else if (format.equals("html")) {
            HtmlGenerator.generate(doc.getContent(), doc.getC
ss());
        }
    }
}
```

This puts all export logic in one class. Adding a new format (Markdown, EPUB) requires modifying this class and adding another branch. The class grows with every new format and becomes increasingly fragile.

Strong Answer

```
public interface DocumentExporter {
    byte[] export(Document document);
```

```
    String getFormatName();
}

public class PdfExporter implements DocumentExporter {
    @Override
    public byte[] export(Document document) {
        PdfDocument pdf = PdfLibrary.createDocument();
        pdf.addContent(document.getContent());
        pdf.applyStyles(document.getStyles());
        return pdf.render();
    }

    @Override
    public String getFormatName() {
        return "pdf";
    }
}
```

Each format is its own class. A `Map<String, DocumentExporter>` routes format names to exporters. Adding Markdown means writing `MarkdownExporter`. Zero existing code changes.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 20: Composition vs Inheritance: The Core OOD Trade-Off

This is the single most important design trade-off in object-oriented design, and interviewers test it directly. "Favor composition over inheritance" is not a slogan to memorize. It is a design principle with concrete technical reasons behind it.

What Each Approach Means Structurally

Inheritance creates an "is-a" relationship. The subclass is a specialized version of the parent. The child inherits all fields, all methods, and all coupling to the parent's implementation.

Composition creates a "has-a" relationship. The class holds a reference to another object and delegates work to it. The composed object can be swapped, configured, or replaced without affecting the composing class.

Why Composition Is Usually Preferred

Dimension	Inheritance	Composition
Coupling	Tight (child depends on parent internals)	Loose (depends only on interface)
Flexibility	Fixed at compile time	Swappable at runtime
Reuse granularity	All or nothing (inherits everything)	Selective (compose only what you need)
Testing	Must construct full parent hierarchy	Can mock composed dependencies
Multiple behaviors	Java allows only single inheritance	Can compose multiple behaviors freely
Fragile base class risk	Yes	No

Inheritance locks you into a hierarchy at compile time. Composition lets you assemble behavior at runtime from interchangeable parts.

Interview Relevance: When you choose inheritance in an LLD interview, the interviewer will likely ask "why not composition?" Have a concrete reason ready. When you choose composition, stating "I chose composition here because the relationship is 'has-a' and I want the flexibility to swap implementations" is a strong signal.

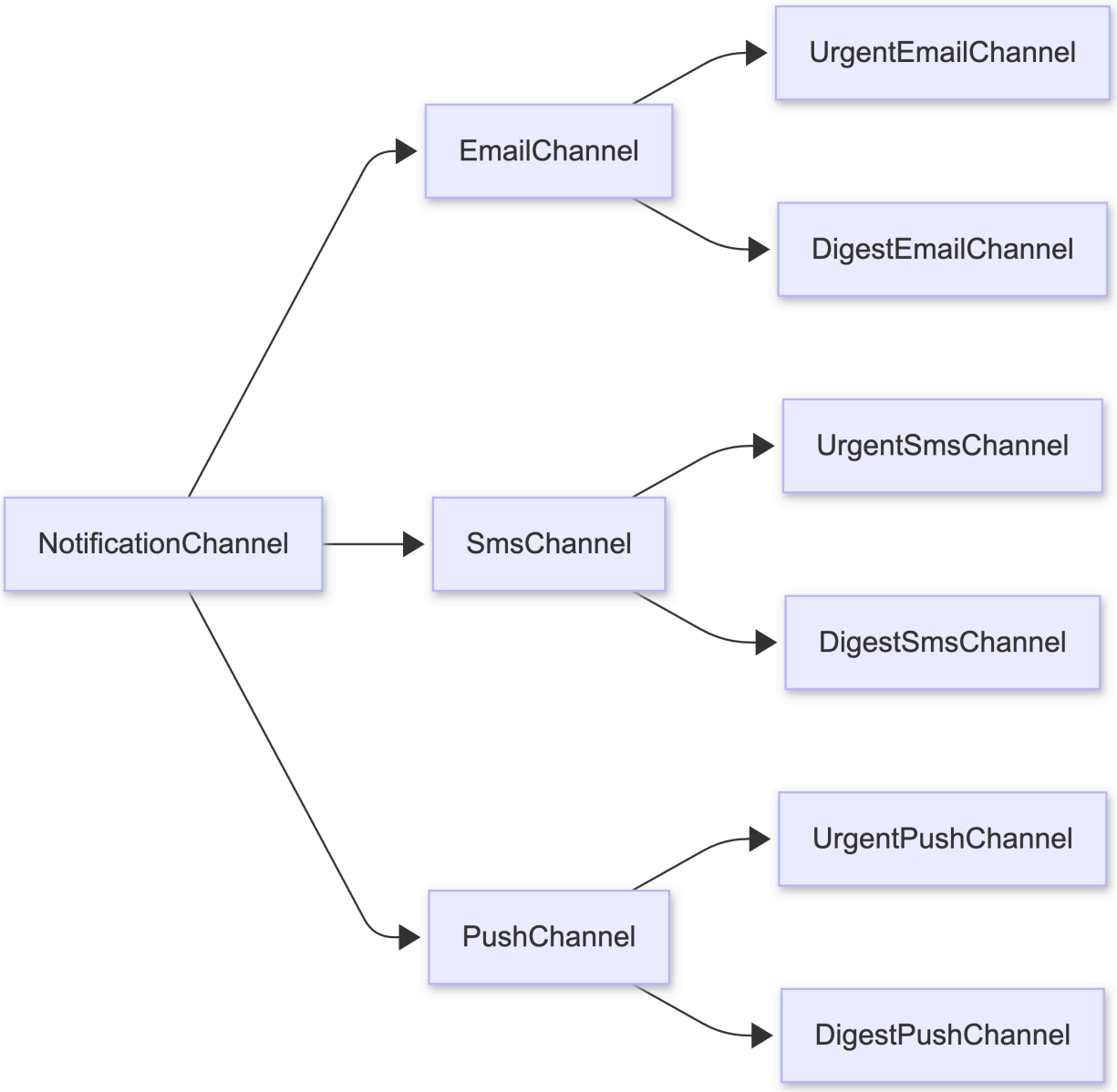
Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 21: Where Inheritance Breaks and Composition Saves You

Consider a notification system for a platform like Slack. You have multiple channel types (email, SMS, push) and multiple priority levels (urgent, normal, digest). Watch what happens with each approach.

Inheritance Approach: Class Explosion



Six leaf classes for two dimensions (3 channels x 2 priorities). Adding a third priority level (scheduled) adds three more classes. Adding a fourth channel (WhatsApp) adds three more. The class count grows as channels multiplied by priorities. This is the classic "inheritance explosion" problem.

Composition Approach: Combine Behaviors

```
public class NotificationSender {
    private final DeliveryMethod deliveryMethod;
    private final PriorityHandler priorityHandler;
```

```
    private final MessageFormatter formatter;

    public NotificationSender(DeliveryMethod deliveryMethod,
                              PriorityHandler priorityHandle
r,
                              MessageFormatter formatter) {
        this.deliveryMethod = deliveryMethod;
        this.priorityHandler = priorityHandler;
        this.formatter = formatter;
    }

    public void send(String userId, String message) {
        String formatted = formatter.format(message);
        priorityHandler.schedule(
            () -> deliveryMethod.deliver(userId, formatted)
        );
    }
}
```

Adding WhatsApp means writing one new `DeliveryMethod` implementation. Adding a "scheduled" priority means writing one new `PriorityHandler` implementation. The class count grows as channels plus priorities, not channels multiplied by priorities.

Interview Tip: When an interviewer sees your class diagram has two or more independent dimensions of variation (channel type AND priority, vehicle type AND fuel type, report format AND delivery schedule), point out that inheritance forces you to pick one hierarchy while composition lets you vary each dimension independently. This is one of the strongest signals of design maturity in an LLD round.

Interview Relevance: "Favor composition over inheritance" is tested directly. Interviewers want to see that you understand when inheritance creates coupling problems, not just that you have memorized the phrase.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 21B: Composition vs Inheritance: Decision Framework

Use this decision framework during your LLD interview when choosing between inheritance and composition.

Choose Inheritance When

All three of these conditions are true simultaneously:

1. The relationship is genuinely "is-a" (a `SavingsAccount` is a `BankAccount`)
2. The subclass uses most or all of the parent's behavior, not just one method
3. The parent class was designed for extension with clear abstract methods or hooks

If any one of these conditions is false, composition is the safer choice.

Choose Composition When

Any one of these conditions is true:

1. The relationship is "has-a" or "uses-a" (an `Order` has a `PaymentProcessor`)
2. You need to combine behaviors from multiple sources
3. You want to swap behavior at runtime (different strategies, different implementations)
4. You see two or more independent dimensions of variation
5. You want the composed object to be independently testable

Quick Decision Table

Signal in Requirements	Approach	Reasoning
"Different types of X" with shared core behavior	Inheritance (cautiously)	True "is-a" with a common base
"Different types of X" with different behaviors	Composition + Interface	Plug-in architecture
"X can do Y in multiple ways"	Composition + Strategy	Swappable algorithms
"X needs capabilities from A and B"	Composition	Java only allows single inheritance

Signal in Requirements	Approach	Reasoning
"X must work with different Z implementations"	Composition + Dependency Injection	Loose coupling for testability

Interview Relevance: Stating your decision criteria out loud during an LLD interview ("I chose composition here because there are two independent axes of variation") demonstrates that you are making a deliberate trade-off, not just defaulting to the first approach that comes to mind.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 22: Abstract Classes vs Interfaces

Both abstract classes and interfaces define contracts that subclasses or implementing classes must fulfill. The difference is in what they can provide beyond the contract.

Abstract Class

An abstract class can hold state (instance fields), provide concrete method implementations, and define constructors. It establishes a "is-a" relationship and is used when subclasses share common state and behavior.

```
public abstract class DatabaseRepository {
    protected final DataSource dataSource;

    public DatabaseRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    protected Connection getConnection() throws SQLException
{
        return dataSource.getConnection();
    }

    public abstract List<Map<String, Object>> findAll();
```

```
    public abstract void save(Map<String, Object> entity);
}
```

Interface

An interface defines a pure contract with no state. Since Java 8, interfaces can have `default` methods (concrete methods), but they still cannot hold instance fields. A class can implement multiple interfaces.

```
public interface Cacheable {
    String getCacheKey();
    Duration getTimeToLive();
}

public interface Auditable {
    String getLastModifiedBy();
    Instant getLastModifiedAt();
}

public class Product implements Cacheable, Auditable {
    // implements methods from both interfaces
}
```

Key Differences

Dimension	Abstract Class	Interface
Instance fields	Yes	No
Constructors	Yes	No
Multiple inheritance	No (single parent only)	Yes (implement many)
Default methods	Always supported	Since Java 8
Access modifiers on methods	Any	Public (effectively)
Relationship	"Is-a" (strong)	"Can-do" (capability)
State sharing	Can share state across subclasses	Cannot share state

Interview Relevance: Interviewers often ask "why did you use an interface here instead of an abstract class?" or vice versa. Having a clear decision framework

(next slide) lets you answer with concrete technical reasoning.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 22B: Abstract Classes vs Interfaces: Decision Framework

Choose an Interface When

1. You are defining a capability that multiple unrelated classes can have (`Serializable`, `Comparable`, `Cacheable`)
2. A class needs to conform to multiple contracts simultaneously
3. You want maximum flexibility for implementors (no forced inheritance hierarchy)
4. You are defining a contract for dependency injection (the most common case in LLD interviews)

Choose an Abstract Class When

1. Subclasses share state (common fields like `dataSource`, `logger`, `configuration`)
2. You want to provide a partial implementation with template methods that subclasses customize
3. You need constructors to enforce initialization logic
4. The relationship is a genuine shared-implementation hierarchy (all database repositories need a connection)

Common Mistake in Interviews

Using an abstract class when an interface would be better is the more frequent error. Candidates default to abstract classes because they want to share code, but this forces a single-inheritance hierarchy that limits future flexibility.

The Java 8+ Gray Area

Since Java 8, interfaces can have `default` methods, which means interfaces can provide shared behavior without requiring an abstract class. This blurs the line, but the rule remains: if you need shared state (fields), use an abstract class. If you

only need shared behavior (methods), an interface with default methods is usually sufficient.

Decision Shortcut

Need shared fields? Abstract class. Everything else? Interface.

Interview Tip: In an LLD interview, default to interfaces for defining contracts between your classes. Only introduce an abstract class when you can point to specific shared state that subclasses need to inherit. This shows the interviewer that you are not creating unnecessary coupling.

Interview Relevance: The abstract class vs interface decision appears in almost every LLD interview. Interviewers expect you to justify your choice with a specific reason, not just a preference.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 22C: Note It Down: Key OOP Design Decisions

These are the decision points interviewers evaluate most frequently. Write them down or screenshot this slide for quick reference.

The Five Key OOP Decisions in Every LLD Interview

1. **Encapsulation check:** For every field, ask "should external code be able to change this directly?" If the answer requires a condition or rule, expose a behavior method instead of a setter.
2. **Abstraction check:** For every dependency, ask "if I swapped the implementation, would calling code need to change?" If yes, introduce an interface.
3. **Inheritance vs Composition:** Default to composition. Use inheritance only when the relationship is truly "is-a" AND the subclass uses most of the parent's behavior AND the parent was designed for extension.
4. **Interface vs Abstract Class:** Default to interfaces. Use abstract classes only when subclasses share state (fields), not just behavior.

5. **Polymorphism check:** For every `if/else` or `switch` on type, ask "could this be replaced with polymorphic dispatch?" If a method behaves differently based on the type of an object, that behavior belongs in the object, not in the caller.

Quick Reference Table

Smell in Your Design	Likely Fix
Getters and setters on every field	Replace with behavior methods
`if/else` chain on object type	Extract interface, use polymorphism
Class inherits but only uses 1-2 parent methods	Switch to composition
Abstract class with no shared fields	Convert to interface
Calling code knows implementation details	Introduce abstraction layer

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 23: OOP Interview Practice and Common Mistakes

Practice Scenario

You are designing a ride-sharing system like Uber. The system supports multiple vehicle types (sedan, SUV, luxury) and multiple pricing strategies (flat rate, surge pricing, subscription discount). The interviewer asks: "Walk me through your class design."

Pause the video and sketch your class design for 90 seconds before continuing.

Key Design Decisions the Interviewer Evaluates

The vehicle type and pricing strategy are two independent dimensions of variation. This is the composition signal from Slide 21. A strong answer separates them into two interface hierarchies composed together, not an inheritance tree of `SurgePricingSedanRide` and `FlatRateLuxuryRide`.

Common OOP Mistakes in LLD Interviews

Mistake	Why It Is Wrong	Stronger Approach
Getters and setters on every field	Exposes internal state, no invariant protection	Behavior methods that enforce business rules
Using inheritance for code reuse only	Creates coupling without a genuine "is-a" relationship	Composition with delegation
One massive class with all logic	Violates encapsulation and makes testing impossible	Separate classes with focused responsibilities
`if/else` on type strings	Fragile, not extensible, misses polymorphism	Interface with implementations per type
Abstract class where interface suffices	Forces single inheritance, limits flexibility	Interface with default methods if shared behavior is needed
Exposing internal collections	Callers can modify state without validation	Return unmodifiable copies or expose behavior methods

Interview Relevance: This table maps directly to what interviewers write in their feedback. Each "Stronger Approach" column is what earns a "Strong Hire" signal.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 24: Section Summary and Cheat Sheet

Concepts Covered

Concept	One-Line Summary
Encapsulation	Control access to state through behavior methods, not getters/setters
Abstraction	Hide implementation complexity behind clean interfaces
Abstraction Leaks	When implementation details escape through the interface
Inheritance	"Is-a" relationship with full coupling to the parent class
Fragile Base Class	Subclass breaks when parent's internal implementation changes
Runtime Polymorphism	Same method call, different behavior based on actual object type

Concept	One-Line Summary
Compile-Time Polymorphism	Method overloading, resolved by the compiler
Composition over Inheritance	"Has-a" with loose coupling beats "is-a" with tight coupling in most cases
Abstract Class	Shared state + partial implementation for a genuine hierarchy
Interface	Pure contract, multiple implementation, maximum flexibility

Common Interview Mistakes

Mistake	Stronger Approach
"I used getters and setters to encapsulate the data"	"I exposed behavior methods that enforce business rules"
"I used inheritance because the child reuses parent code"	"I used composition because the relationship is 'has-a' and I need runtime flexibility"
"I used an abstract class to define the contract"	"I used an interface because there is no shared state, and implementors may need multiple contracts"
"I added a switch statement for each new type"	"I used polymorphism so adding a new type is a new class, not a code change"

Key Points to Remember

Principle	Key Test
Encapsulation	"Can external code violate an invariant?" If yes, encapsulation is broken
Abstraction	"Would swapping the implementation require caller changes?" If yes, abstraction is leaking
Inheritance	"Is this truly is-a, and does the child use most of the parent?" If not, use composition
Polymorphism	"Is there an if/else on type?" If yes, extract an interface
Interface vs Abstract Class	"Is there shared state?" If no, use an interface

How This Connects to Other Sections

- **Section 1 (Mental Model):** This section covered the "class design clarity" and "correct use of OOP" criteria from the evaluation rubric.

- **Section 3 (SOLID):** SOLID principles build directly on OOP. SRP refines encapsulation. OCP formalizes polymorphism for extensibility. LSP constrains inheritance. ISP refines abstraction. DIP enforces composition over concrete dependencies.
- **Sections 4-6 (Design Patterns):** Every design pattern is built on OOP primitives. Factory uses polymorphism. Strategy uses composition. Observer uses interfaces. Understanding OOP deeply makes every pattern intuitive rather than memorized.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn